

Compulsory Assignment 5: Sentiment Analysis

Please fill out the the group name, number, members and optionally the name below.

Group number: 26

Member 1: Milad Tootoonchi

Member 2: Makka Dulgaeva

Member 3: Sania Minaipour

Assignment Submission

To complete this assignment answer the questions in this notebook and write the code required to implement the relevant models. However, we recommend that you start early. This assignment has subsections, each of which build upon the last. This assignment is completed by doing the following.

- Submit notebook as an .ipynb file to canvas.
- Submit notebook as an .pdf file to canvas.

Your submission must correctly implement the full pipeline to analyse the sentiment using a RNN and BERT models.

NOTE: It is recommended to use Google Colab with GPU enabled for CA5 tasks. Keras and TensorFlow (version ≥ 2) will automatically use the GPU if one is available

- Read tasks thoroughly before answering.
- Make sure to answer all tasks.
- Ensure all code cells are run.
- Ensure all figures are visible in the PDF.
- Provide a brief explanation of how your code works

How to Use Google Colab for CA5

To work on the CA5 assignment in Google Colab, follow these steps:

1. Connect Google Colab to Google Drive

Your CA5 files should be stored in Google Drive so Colab can access them.
Run the following code to mount your Drive:

Change Runtime to GPU

To use GPU in Google Colab:

1. Go to the top menu and click **Runtime** → **Change runtime type**.
2. In the pop-up window, set **Hardware accelerator** to **GPU**.
3. Click **Save**.

Now your notebook will run on a GPU backend.

Note: I recommend using the regular runtime first, as your GPU usage is limited. Write and test your code in the regular runtime to ensure everything works correctly before switching to the GPU for model training.

Introduction

In this assignment, you will analyze whether a movie review is positive or negative. In Task 1, you will use a Recurrent Neural Network (RNN) to perform the analysis. In Task 2, you will be introduced to the encoder block of the Transformer architecture, commonly known as BERT. Finally, you will compare the results from both approaches and draw a conclusion.

Note: BERT stands for "Bidirectional Encoder Representations from Transformers"

About the Dataset

The IMDb dataset is a collection of 50,000 reviews from the Internet Movie Database (IMDb). The reviews are labeled as either positive or negative.

The IMDb dataset is a binary sentiment analysis dataset for natural language processing or text analytics.

IMDb is a rich source of film data that includes cast and crew lists, movie release dates, box office information, plot summaries, trailers, actor and director biographies, and other trivia. Information on IMDb comes from a variety of sources, such as filmmakers, film studios, on-screen credits, and other official sources.

Task 0: Load and Inspect the Data

Preprocess

Task 1: RNN

1.1 Vectorize inputs

Task 1.1 Create a simple RNN and an LSTM model

Compile and print out summary

Task 1.2: Train your model

Task 1.3 Evaluate your models

Task 2: BERT

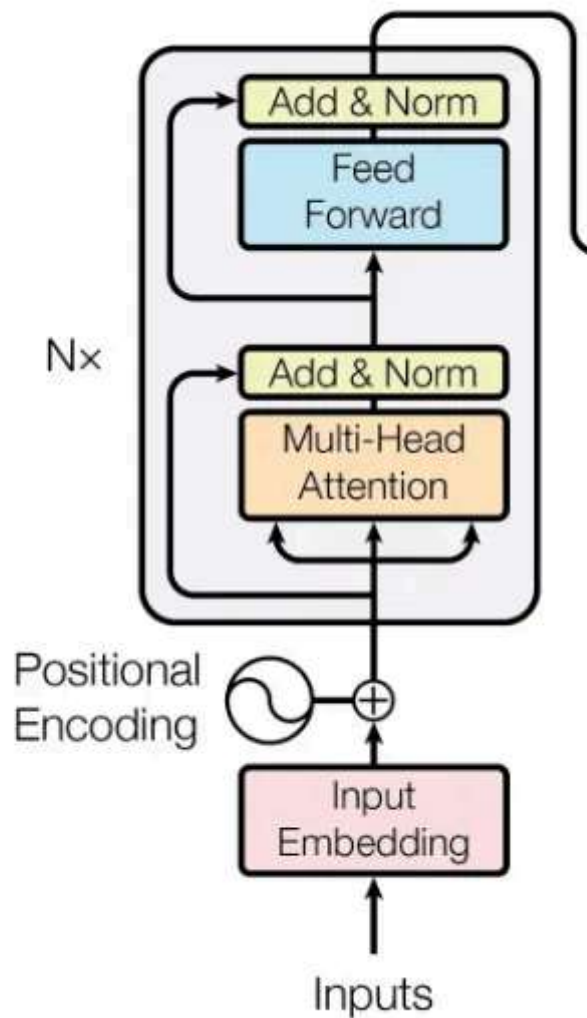
Task 2.0: Install the transformers package from HuggingFace

Hugging Face is an American company and an open-source community that provides tools, pre-trained machine learning models, and datasets, primarily focused on natural language processing (NLP) and other AI fields like computer vision and audio

Note: You may need to restart the runtime after installing the packages. Then, mount your Google Drive and rerun Task 0.

Task 2.1: Load the distilbert model

In this part of the assignment, we will use a smaller and faster version of BERT called DistilBERT. It is significantly more lightweight and trains much faster than the original BERT model. Here is the illustration of its architecture:



Task 2.2: Split the training data to Validation and training

Task 2.3: Tokenize

You have to tokenize your inputs again, use the AutoTokenizer that we imported earlier.

Create datasets

This step wraps your input features (enc_train, e.g., input_ids and attention_mask) and labels (y_train) into a TensorFlow-native streaming dataset.

The TensorFlow Dataset API provides an efficient way to feed data to your model:

- Streams data efficiently from RAM or disk.
- Performs batching, shuffling, and preprocessing in parallel on the CPU.

- Keeps the GPU busy by prefetching the next batch while training.
- Integrates seamlessly with Hugging Face's encoded token dictionaries.

Task 2.4: Create and Compile model

Task 2.5: Train your model

Task 2.6: Evaluate

Task 3: Compare

3.1: Compare the results of the two models. What could explain the differences in their performance?

Answer:

The LSTM did better than the simple RNN because it can remember more information from earlier in the sentence. The simple RNN tends to forget things as it goes along, so it struggles with longer text. The BERT model did worse, probably because it was not trained for enough time or maybe the setup was not quite right. BERT usually needs more fine tuning and the right tokenizer to work well.

3.2: Why did we need to tokenize the inputs again using a different tokenizer in Task 2.3?

Answer:

We had to tokenize again because BERT uses its own special kind of tokenizer. It splits words into smaller pieces and adds extra tokens that BERT needs to understand the text. The tokenizer we used before with the RNN and LSTM just turned words into numbers, which would not work with how BERT was trained.

3.3: Briefly explain the main difference between RNNs and Transformer architectures.

Answer:

The main difference is that RNNs read text one word at a time, while transformers like BERT look at all the words at once. RNNs are slower and can forget older information, but transformers can find connections between words no matter how far apart they are. This makes transformers more powerful for language tasks.

Task 4: Reading Exercise (Optional)

If you truly want to understand the logic behind ChatGPT and the motivation for Transformers, I encourage you to read the 2017 paper "[Attention Is All You Need.](#)" In this paper, engineers from Google explain in detail how the architecture was built and why it represents such a major breakthrough.